

ASSESSMENT : 7

STUDENT NAME : K.SRIJA

✓ 1. Define a class Car with attributes and a method to display details.

A class is a blueprint for creating objects. A class contains attributes (data) and methods (functions). Objects created from a class can access these attributes and methods.

```
class Car:
    def __init__(self, brand, model, year):
        self.brand = brand
        self.model = model
        self.year = year

    def display_details(self):
        print("Brand:", self.brand)
        print("Model:", self.model)
        print("Year:", self.year)

car1 = Car("Toyota", "Corolla", 2022)
car1.display_details()
```

```
Brand: Toyota
Model: Corolla
Year: 2022
```

✓ 2. Explain the purpose of the **init** method.

The **init** method is called automatically whenever an object is created. It is known as the constructor and is used to initialize the object's attributes.

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

student1 = Student("Alice", 20)

print(student1.name)
print(student1.age)
```

```
Alice
20
```

✓ 3. Create a subclass ElectricCar that inherits from Car.

Inheritance allows one class to acquire the properties and methods of another class. The child class inherits all the features of the parent class.

```
class Car:
    def __init__(self, brand):
        self.brand = brand

    def display(self):
        print("Brand:", self.brand)

class ElectricCar(Car):
    def __init__(self, brand, battery):
        super().__init__(brand)
        self.battery = battery

    def display_battery(self):
        print("Battery Capacity:", self.battery)

car = ElectricCar("Tesla", "75 kWh")

car.display()
car.display_battery()
```

Brand: Tesla
Battery Capacity: 75 kWh

✓ 4. Demonstrate Method Overriding in inheritance.

Method overriding occurs when a child class provides its own implementation of a method already defined in the parent class.

```
class Animal:
    def sound(self):
        print("Animal makes a sound")

class Dog(Animal):
    def sound(self):
        print("Dog barks")

obj = Dog()
obj.sound()

Dog barks
```

✓ 5. Explain Encapsulation by creating private attributes using __.

Encapsulation is the process of hiding data from direct access. Private attributes are created using double underscores (__).

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.__salary = salary

    def show_salary(self):
        print("Salary:", self.__salary)

emp = Employee("John", 50000)

print(emp.name)
emp.show_salary()

John
Salary: 50000
```

✓ 6. Use @property to create getter and setter methods.

The @property decorator allows controlled access to private variables using getter and setter methods.

```
class Student:
    def __init__(self):
        self.__marks = 0

    @property
    def marks(self):
        return self.__marks

    @marks.setter
    def marks(self, value):
        self.__marks = value

s = Student()

s.marks = 90

print("Marks:", s.marks)

Marks: 90
```

✓ 7. What is Multiple Inheritance? Provide a small code structure.

Multiple inheritance allows a class to inherit from more than one parent class.

```

class Father:
    def skills1(self):
        print("Gardening")

class Mother:
    def skills2(self):
        print("Cooking")

class Child(Father, Mother):
    def skills3(self):
        print("Programming")

c = Child()

c.skills1()
c.skills2()
c.skills3()

```

```

Gardening
Cooking
Programming

```

8. Create a class method using the @classmethod decorator.

A class method belongs to the class rather than an object. It uses cls as the first parameter and is defined using the @classmethod decorator.

```

class College:
    college_name = "ABC College"

    @classmethod
    def show_college(cls):
        print("College Name:", cls.college_name)

College.show_college()

```

```

College Name: ABC College

```

9. Explain the difference between a class method and a static method.

Class Method

- Uses cls parameter
- Can access class variables
- Uses @classmethod

Static Method

- No self or cls parameter
- Cannot directly access class or instance variables
- Uses @staticmethod

```

class Demo:
    college = "XYZ College"

    @classmethod
    def class_method(cls):
        print("Class Method:", cls.college)

    @staticmethod
    def static_method():
        print("Static Method")

Demo.class_method()
Demo.static_method()

```

```

Class Method: XYZ College
Static Method

```

10. Write a try-except-finally block to handle a ZeroDivisionError.

The try block contains code that may produce an error. The except block handles the error, and the finally block executes whether an exception occurs or not.

```
try:
    num1 = 10
    num2 = 0

    result = num1 / num2

    print(result)

except ZeroDivisionError:
    print("Error: Division by zero is not allowed.")

finally:
    print("Program execution completed.")
```

Error: Division by zero is not allowed.
Program execution completed.